Лекція №15 (2 години). JSX, Props та рендеринг списків.

План лекції

1. Компоненти в React: Функціональні компоненти.
2. Синтаксис JSX: суміш HTML та JavaScript.
3. Головні правила написання JSX.
4. Передача даних у компоненти: Props (Властивості).
5. Деструктуризація Props та значення за замовчуванням.
6. Рендеринг списків за допомогою методу `map()`.
7. Важливість атрибута `key` при роботі зі списками.
8. Умовний рендеринг (Тернарний оператор та &&).

Перелік умовних скорочень

- **JSX** (JavaScript XML) — розширення синтаксису JavaScript для написання розмітки.
- **UI** (User Interface) — користувацький інтерфейс.
- **Props** (Properties) — властивості/параметри, які передаються в компонент.
- **DOM** (Document Object Model) — об'єктна модель документа.

Вступ

На минулій лекції ми дізналися, що React будує інтерфейси у вигляді "лего-блоків" — незалежних компонентів. Але як саме створювати ці компоненти?

Історично існували два підходи: створення через Класи (Class Components, старий підхід) та через Функції (Functional Components, сучасний підхід). Сьогодні вся індустрія перейшла виключно на функціональні компоненти, тому що вони простіші для розуміння і містять менше шаблонного (boilerplate) коду.

Окрім компонентів, головною родзинкою React є **JSX** — синтаксис, що дозволяє писати HTML-теги **безпосередньо** всередині JavaScript-коду. Спочатку це виглядає неприродно для розробників, які звикли жорстко відділяти логіку (JS) від розмітки (HTML). Але цей підхід робить створення динамічних інтерфейсів набагато зручнішим.

Мета цієї лекції — навчитися створювати функціональні компоненти, вільно писати JSX-розмітку, передавати дані між компонентами за допомогою Props і динамічно виводити великі масиви даних (наприклад, каталоги товарів).

1. Функціональні компоненти

Сучасний React-компонент — це звичайнісінька JavaScript-функція. Ця функція отримує певні дані (Props) на вхід і завжди **зобов'язана** повернути (`return`) шматок UI-розмітки на вихід.

Два головних правила для компонентів:

1. Назва компонента завжди повинна починатися з **Великої літери** (наприклад, `Button` , а не `button`). Якщо ви напишите з маленької, React подумає, що це звичайний вбудований HTML-тег.
2. Компонент завжди повинен повертати єдиний кореневий елемент.

Приклад компонента `Header.jsx` :

```
// Використовуємо звичайну або стрілочну функцію
function Header() {
  return (
    <header>
      <h1>Вітаємо у React!</h1>
    </header>
  );
}

export default Header; // Експортуємо, щоб використати в інших місцях
```

Застосування створеного компонента в головному файлі (наприклад, `App.jsx`):

```
import Header from "./Header";

function App() {
  return (
    <div>
      <Header /> {/* Викликаємо компонент як HTML-тег, що самозакривається */}
      <main>Основний контент сторінки</main>
    </div>
  );
}
```

2. Синтаксис JSX: суміш HTML та JS

JSX — це синтаксичний цукор (надбудова над JS). Браузер насправді не розуміє JSX. Під капотом інструменти збірки (наприклад, Vite + Babel) перетворюють кожен тег `<h1>Привіт</h1>` на складний і нечитабельний виклик функції `React.createElement('h1', null, 'Привіт')`. JSX створений тільки для того, щоб розробникам було зручно візуально уявляти DOM-дерево.

Найважливіша фішка JSX — **вставка JavaScript всередину HTML**. Щоб вставити будь-який динамічний вираз JS (змінну, результат виклику функції, математичну операцію) у розмітку, достатньо взяти його у **фігурні дужки** `{}`.

```
function UserProfile() {
  const name = "Олег";
  const age = 25;
  const currentYear = new Date().getFullYear(); // 2026

  return (
    <div className="card">
      <h2>Ім'я користувача: {name.toUpperCase()}</h2>
      <p>Вік: {age}</p>
      <p>Рік народження: {currentYear - age}</p>
    </div>
  );
}
```

> *Важливо: У фігурні дужки можна вставляти ТІЛЬКИ вирази (expressions), які повертають значення. (Не можна вставити туди цикл `for` або конструкцію `if...else`).*

3. Головні правила написання JSX

1. **Один кореневий елемент:** JSX мусить повертати єдину цілісну "обгортку".
   - *Неправильно:* `return <h1>...</h1> <p>...</p>;`
   - *Правильно:* обгорнути це у `<div>...</div>`. Якщо ви не хочете плодити "сміттєві" діви створюючи div-soup, використовуйте **React Фрагменти**: порожні теги `<> ... </>`. Вони зникають після рендерингу в браузері.
2. **Всі теги мають бути закритими:** Навіть одинарні теги, які в звичайному HTML можуть не мати слеша (як `img`, `br`, `input`), у JSX зобов'язані закриватися: `<img src="logo.png" />` або `<input type="text" />`.

3. **атрибут `class` змінюється на `className`:** Оскільки "class" — це зарезервоване (ключове) слово в JavaScript (для створення ООП-класів), React використовує атрибут `className` для прив'язки CSS-класів. У фінальному HTML він перетвориться на звичайний `class`.
4. **camelCase для атрибутів:** Усі мультислівні HTML-атрибути пишуться стилем camelCase (наприклад, `onclick` стає `onClick`, `tabindex` стає `tabIndex`).

4. Передача даних: Props (Властивості)

Компоненти були б не дуже корисними, якби вони малювали завжди одне й те саме захардкоджене значення. Нам потрібно мати можливість "передавати налаштування" всередину компонента. Для цього використовуються **Props** (скорочення від Properties).

Props — це об'єкт з даними, які ми передаємо зверху-вниз: від Компонента-Батька до Компонента-Дитини. Синтаксично це виглядає як передача звичайних HTML-атрибутів:

Батьківський компонент `App.jsx`:

```
import WelcomeBox from "./WelcomeBox";

function App() {
  return (
    <div>
      {/* Передаємо пропси name, age та role */}
      <WelcomeBox name="Олена" age={28} role="Адмін" />
      <WelcomeBox name="Ігор" age={19} role="Користувач" />
    </div>
  );
}
```

Зверніть увагу: рядки передаються в лапках `" "`, а числа (або об'єкти, чи масиви) ОБОВ'ЯЗКОВО беруться у фігурні дужки `{28}`.

5. Деструктуризація Props та значення за замовчуванням

Як дитячий компонент приймає ці дані? Він отримує їх як єдиний аргумент-об'єкт `props`.

```
// Варіант 1: Довгий (без деструктуризації)
function WelcomeBox(props) {
```

```
  return (
    <article>
      <p>Привіт, {props.name}!</p>
      <small>Вік: {props.age}</small>
    </article>
  );
}
```

Але в сучасному React всі використовують деструктуризацію безпосередньо в параметрах функції і відразу отримують потрібні змінні. Також тут зручно призначати значення за замовчуванням (`=`), якщо батько раптом забув передати prop.

```
// Варіант 2: Сучасний (з деструктуризацією)
function WelcomeBox({ name, age, role = "Гість" }) {
  return (
    <article>
      <p>
        Привіт, {name} (статус: {role})!
      </p>
      <small>Вік: {age}</small>
    </article>
  );
}
```

> ***Золоте правило Props:*** *Пропси є "Тільки для читання" (Read-Only). Компонент-дитина* ***ніколи*** *не має права змінювати пропси, які йому передали! Це грубе порушення архітектуриReact (і згенерує помилку).*

6. Рендеринг списків за допомогою методу `map()`

Майже у кожному додатку (від стрічки Instagram до списку товарів у Rozetka) доводиться перетворювати масив сирих даних із бекенду на великий список HTML/JSX-елементів. В React це робиться виключно за допомогою стандартного методу масивів `map()` .

Оскільки `map` повертає новий масив і є виразом, ми пишемо його прямо у JSX, всередині фігурних дужок `{}` :

```
const usersData = [
  { id: 1, name: "Анна" },
  { id: 2, name: "Богдан" },
```

```
  { id: 3, name: "Віктор" },
];

function UserList() {
  return (
    <ul>
      {usersData.map((user) => (
        // Зверніть увагу: використовуємо дужки () для повернення JSX, а не {}
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

Замість простого `<li>` ми могли б повертати складний компонент `<UserCard name={user.name} />` і відмалювати тисячі карток трьома рядками коду.

7. Важливість атрибута `key`

Якщо ви виконаєте код вище без атрибута `key`, React виведе в консоль яскраве червоне попередження: `Warning: Each child in a list should have a unique "key" prop.`

Навіщо потрібен key? Коли список великий і динамічний (товари можна видаляти, сортувати, додавати), React-у важко слідкувати за тим, який саме елемент змінився, якщо всі вони є просто тегами `<li>`. Атрибут `key` — це унікальний бейдж (паспорт) для кожного елемента списку. Завдяки ключам алгоритм *Реконсиляції Віртуального DOM* (Diffing) працює блискавично. Якщо ми видалимо другий елемент, React (дивлячись на ключі) зрозуміє саме це видалення і не буде перемальовувати решту 999 елементів списку.

Правила `key`:

1. Ключ має бути стабільно прив'язаним до елемента (не повинен генеруватися випадково функцією `Math.random()`).
2. Найкращий ключ — це унікальний `ID` із бази даних (`id: 12345`).
3. Використовувати індекси масиву (`map((user, index)`) у якості `key` — вважається крайнім антипатерном, оскільки при сортуванні або видаленні з масиву індекси змістяться і React заплутається і зрендерить хибні дані. Використовувати індекси можна **тільки** якщо список суто статичний і ніколи не мінятиметься.

8. Умовний рендеринг

А що, якщо ми хочемо показувати компонент `<AdminPanel />`, якщо `isAdmin === true`, і загальний `<ErrorText />`, якщо `false`? У React не можна просто вставити класичний `if` всередину JSX-розмітки `return (...)`.

Є два улюблені способи умовного показу в React (вбудовуються у `{}`):

1. Тернарний оператор (Умова ? Так : Ні) Використовується, коли є два альтернативних варіанти.

```
function Dashboard({ isAdmin }) {
  return (
    <div>
      {isAdmin ? <AdminPanel /> : <p>Доступ заборонено (ви не адмін).</p>}
    </div>
  );
}
```

2. Логічне АБО (Умова && Так) Використовується, коли у разі негативної умови взагалі **нічого** не потрібно малювати. Якщо перша частина `true`, друга відмальовується. Якщо `false`, друга частина ігнорується.

```
function ShoppingCart({ cartItems }) {
  // Якщо масив не пустий (>0) - показуємо кнопку "Оплатити"
  return (
    <div>
      <ul>{/*...тут товари...*/}</ul>

      {cartItems.length > 0 && <button>Оплатити замовлення</button>}
    </div>
  );
}
```

Висновки

1. Сучасні React-додатки будуються виключно на функціональних компонентах. Компонент — це JS-функція, що починається з великої літери та повертає JSX.
2. JSX розмиває межі між логікою та розміткою: ви пишете HTML-подібні структури прямо у коді і вставляєте в них JS-змінні через `{}`.

3. Коренева обгортка (Fragment `<></>`), атрибут `className` та самозакриття всіх тегів — базові вимоги JSX-синтаксису.
4. `Props` — це механізм потоку даних згори-донизу. Через пропси батьки "налаштовують" дітей. Пропси в дитині є Read-Only (змінювати їх не можна).
5. Виведення десятків і сотень елементів (масиву даних) з бекенду вирішується у JSX використанням декларативного методу `.map()`.
6. Створюючи списки з `map`, завжди необхідно передавати унікальний стабільний `key` до головного вузла обгортки (зазвичай ID з бази даних). Це критично для продуктивності React.
7. Відображення і приховування елементів в залежності від логіки (Умовний рендеринг) здійснюється виключно за допомогою тернарного оператора `? :` або логічного "І" `&&` безпосередньо у фігурних дужках.

Джерела

1. [React Docs: Writing Markup with JSX] — офіційна документація щодо правил JSX.
2. [React Docs: Passing Props to a Component]
3. [React Docs: Rendering Lists] — чому `key` важливий.
4. [Kent C. Dodds: Why React Elements Require a Key] — глибоке занурення в те, як Diffing алгоритм використовує ключі.

Запитання для самоперевірки

1. Чому всі React-компоненти обов'язково повинні називатися з великої літери?
2. Для чого існують React Фрагменти (`<> </>`)?
3. Дайте визначення терміну `Props`. У якому напрямку між компонентами можлива їхня передача? Чи має право син-компонент поміняти те, що йому передали?
4. Напишіть приклад коду, як за допомогою деструктуризації витягнути з пропсів параметри `title` та `color` (де `color` за замовчуванням буде "black").
5. Чому React змушує нас використовувати атрибут `className` замість класичного HTML-атрибута `class`?
6. За допомогою якого методу масиву здійснюється масове створення (рендерінг) компонентів у вигляді списку?
7. Чому використання випадкових чисел (на кшталт `Math.random()`) або індексів масиву у якості `key` в списках — це погана практика і може викликати страшні баги? Яке джерело для значення атрибута `key` є еталонним?
8. Що таке умовний рендеринг і які два оператори JavaScript використовуються для відображення або приховування елементів у JSX на його основі?